



NexusOdonto

SISTEMA INTEGRAL DE GESTIÓN ODONTOLÓGICA

Documentación Técnica del Backend

Arquitectura Limpia en Capas (.NET 10), Persistencia Relacional Oracle Database, Control de Acceso RBAC, Protocolos Clínicos y Guía para Sustentación

.NET 10 (C# 14)

ASP.NET Core Web API

Oracle Database 21c / 23ai

Entity Framework Core 10

Clean Architecture (4 Capas)

Microsoft SignalR Core

JWT Bearer + Google OAuth 2.0

FluentValidation 12

Serilog Structured Logging

Polly HTTP Resilience

BCrypt.Net Security

RFC 7807 ProblemDetails

Equipo de Desarrollo Backend NexusOdonto

Documento Oficial de Arquitectura, Contratos de Integración y Reglas de Negocio · Octubre 2026

Índice General de Secciones

1. Alcance y guía de lectura	Pág. 2	10. Agenda médica, disponibilidad y reglas horarias	Pág. 10
2. Propósito y organización funcional	Pág. 2	11. Odontograma anatómico FDI y expediente clínico	Pág. 11
3. Catálogo completo de bibliotecas NuGet (.csproj)	Pág. 3	12. Comunicación en tiempo real (SignalR) y presencia	Pág. 12
4. Arquitectura de software en 4 capas	Pág. 4	13. Persistencia Oracle, transacciones y seeders	Pág. 13
5. Pipeline HTTP, middlewares y excepciones	Pág. 5	14. Value Objects y diseño guiado por el dominio (DDD)	Pág. 13
6. Modelo de identidad y autenticación	Pág. 6	15. Validaciones declarativas y resiliencia Polly	Pág. 14
7. Cuentas oficiales de prueba para evaluación	Pág. 6	16. Seguridad, confidencialidad y enmascaramiento	Pág. 14
8. Control de acceso RBAC y matriz de permisos	Pág. 7	17. Guía técnica para sustentación ante jurados	Pág. 15
9. Catálogo completo de endpoints REST (56 controladores)	Pág. 8	18. Referencias cruzadas de código fuente	Pág. 16

1. Alcance y Guía de Lectura

El backend de **NexusOdonto** es el núcleo transaccional del sistema. Implementado en **.NET 10** con **C# 14**, expone una API RESTful y un Hub WebSocket (SignalR) diseñados bajo principios de *Clean Architecture*, persistencia en **Oracle Database 21c/23ai** y control de acceso RBAC. Este documento constituye el referente técnico oficial para la evaluación y sustentación del proyecto.

2. Propósito y Organización Funcional por Dominios

El backend administra la operación de la clínica estructurada en 10 dominios funcionales desacoplados:

Dominio	Responsabilidad y Funcionalidades	Entidades Principales
1. Identidad & Seguridad	Cuentas de usuario, hashing BCrypt, JWT, OAuth Google, refresh tokens y roles RBAC.	UserEntity, RoleEntity, PermissionEntity, SessionEntity
2. Personas & Directorio	Datos filiatorios legales, pacientes, odontólogos, especialidades médicas y licencias.	PersonEntity, PatientEntity, EmployeeEntity, ProfessionalEntity
3. Agenda & Citas	Programación de citas, evaluación de impacto horario y asignación de doctor libre.	AppointmentEntity, AvailabilityEntity, ClinicSettingsEntity
4. Expediente Clínico	Atenciones médicas, diagnósticos CIE-10, evoluciones, antecedentes y recetas.	ClinicalAttentionEntity, ClinicalHistoryEntity, DiagnosisEntity
5. Odontograma FDI	Registro anatómico de 52 piezas FDI con 5 superficies vectoriales y hallazgos clínicos.	OdontogramEntity, OdontogramToothEntity, DentalFindingEntity
6. Catálogos Maestros	26 catálogos normalizados con códigos canónicos e identificadores inmutables.	ServiceEntity, DocumentTypeEntity, AppointmentStatusEntity...
7. Tiempo Real & Presencia	Hub SignalR para difusión de citas y monitor de presencia de personal en línea.	HubNotificationEntity, PresenceTracker, NexusNotificationHub
8. Notificaciones WhatsApp	Encolamiento y envío automatizado de confirmaciones mediante API externa resiliente.	WhatsAppNotificationEntity, EvolutionApiClient
9. Soporte Asistencial	Conversaciones de pacientes, mensajes históricos y tickets de soporte clasificados.	ChatbotConversationEntity, ChatbotMessageEntity, SupportTicketEntity
10. Auditoría & Trazabilidad	Registro inmutable de accesos, inicios de sesión y modificaciones críticas en Oracle.	AuditEventEntity, AuditEventTypeEntity

AD001 · Administrador Acceso total: usuarios, matriz RBAC, auditoría y configuración clínica.	OD001 · Odontólogo Panel clínico: agenda médica, historia clínica, CIE-10 y odontograma FDI.	RC001 · Recepcionista Gestión operativa: agendamiento, reprogramación y recepción de pacientes.	PA001 · Paciente Portal de autoservicio: consulta de citas, atenciones y odontograma.
---	--	---	---

3. Catálogo Completo de Bibliotecas y Paquetes NuGet (.csproj)

A continuación se detalla cada paquete NuGet importado en los proyectos de la solución (.NET 10), su versión y su justificación técnica:

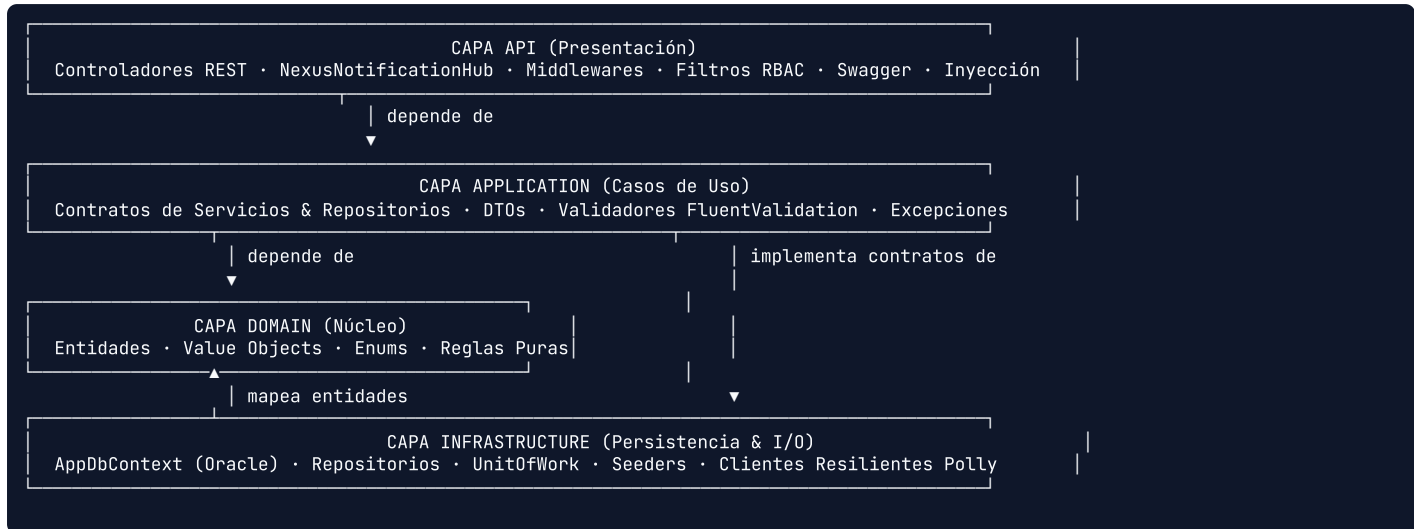
Paquete NuGet	Versión	Proyecto	Propósito y Justificación Técnica en el Sistema
Oracle.EntityFrameworkCore	10.23.26300	Infrastructure	Proveedor oficial de EF Core para Oracle Database 21c/23ai. Gestiona mapeos relacionales y dialecto SQL Oracle.
Microsoft.EntityFrameworkCore	10.0.11	Infrastructure	ORM central para consultas LINQ fuertemente tipadas, seguimiento de cambios y transacciones atómicas.
Microsoft.EntityFrameworkCore.Design	10.0.11	Api / Infra	Herramientas de diseño en tiempo de desarrollo para scaffolding y generación de migraciones relacionales.
Microsoft.EntityFrameworkCore.Tools	10.0.11	Api	Comandos de consola CLI para aplicar migraciones (<code>dotnet ef database update</code>) sobre Oracle.
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.11	Api	Middleware de autenticación para validar tokens Bearer JWT en peticiones HTTP y WebSocket SignalR.
Microsoft.AspNetCore.Authentication.Google	10.0.11	Api	Proveedor de autenticación federada OAuth 2.0 con Google para login e integración de cuentas.
System.IdentityModel.Tokens.Jwt	8.22.0	Api / App	Creación, firma HMAC-SHA256 y validación criptográfica de tokens de acceso y tokens de enlace Google.
Microsoft.IdentityModel.Tokens	8.22.0	Api	Manejo de llaves simétricas (<code>SymmetricSecurityKey</code>) y parámetros de validación de seguridad.
FluentValidation	12.1.1	Application	Motor de reglas de validación declarativas fuertemente tipadas para DTOs, desacoplado de controladores.
FluentValidation.DependencyInjectionExtensions	12.1.1	Application	Escaneo y registro automático de todos los validadores de la capa de aplicación en el contenedor de IoC.
BCrypt.Net-Next	4.2.0	App / Infra	Hashing seguro con sal adaptativa (factor 11) para contraseñas de usuarios contra ataques de fuerza bruta.
Mapster & Mapster.DependencyInjection	10.0.12	Api	Mapeador de alto rendimiento compilado en IL para transformar entidades a DTOs sin sobrecarga de memoria.
Microsoft.Extensions.Http.Resilience	10.9.0	Infrastructure	Políticas de resiliencia con Polly (reintentos exponenciales, timeout y circuit breaker) en llamadas HTTP.
Serilog.AspNetCore & Sinks (Console/File)	10.0.0 / 6.1.1	Api	Registro estructurado en JSON con correlación de peticiones, logs de auditoría y salidas duales.
Swashbuckle.AspNetCore	10.2.3	Api	Generación automática de especificación OpenAPI 3.0 e interfaz Swagger UI con soporte Bearer JWT.
DotNetEnv	3.2.0	Api / Infra	Cargador de variables de entorno desde archivos <code>.env</code> locales para desarrollo y despliegue seguro.

Garantía de Compatibilidad .NET 10

Todos los paquetes han sido validados y compilados sin advertencias ni conflictos bajo el SDK 10.0.302 de .NET.

4. Arquitectura de Software en 4 Capas (Clean Architecture)

El backend implementa formalmente **Clean Architecture**. Las dependencias fluyen estrictamente hacia adentro:



1. Capa Domain (NexusOdontoBackend_Api.Domain)

Núcleo puro sin dependencias: Contiene las entidades relacionales (45 entidades entre principales y catálogos), Value Objects inmutables (`Price`, `TimeOfDay`, `ProfessionalLicense`) y tipos de valor. No conoce ORMs ni bases de datos.

2. Capa Application (NexusOdontoBackend_Api.Application)

Orquestación de casos de uso: Define contratos de interfaces (`IAppointmentService`, `IUserRepository`), DTOs fuertemente tipados, validadores con `FluentValidation` y excepciones semánticas (`ValidationException`, `ConflictException`).

3. Capa Infrastructure (NexusOdontoBackend_Api.Infrastructure)

Adaptadores de persistencia: Implementa `AppDbContext` para Oracle 21c/23ai, configuraciones Fluent API, repositorios genéricos y específicos, transacciones con Unit of Work, seeders y clientes HTTP con Polly.

4. Capa Api (NexusOdontoBackend_Api.Api)

Punto de entrada HTTP/SignalR: Contiene los 56 controladores REST, el Hub de SignalR (`NexusNotificationHub`), middleware global de excepciones, filtros de autorización RBAC y configuración Swagger.

Principio de Inversión de Dependencias (DIP)

Los controladores de la capa **Api** únicamente interactúan con interfaces de **Application** (`IPatientService`). Los servicios interactúan con repositorios mediante contratos (`IPatientRepository`). Las implementaciones concretas residen en **Infrastructure** y se inyectan en `Program.cs` mediante `AddApplication()` y `AddInfrastructure()`.

5. Pipeline HTTP, Middlewares y Manejo de Errores (RFC 7807)

El pipeline de ejecución de ASP.NET Core está configurado en `Api/Program.cs` para garantizar una secuencia estricta de procesamiento, seguridad y trazabilidad:

```
Petición Cliente HTTP / WebSocket
├─ 1. ForwardedHeadersMiddleware (Preserva IP, protocolo HTTPS y host desde proxy Nginx)
├─ 2. GlobalExceptionHandler (Captura global de errores y serialización RFC 7807)
├─ 3. SwaggerUI (Documentación interactiva OpenAPI habilitada en desarrollo y producción)
├─ 4. HttpsRedirection (Redirección obligatoria a tráfico seguro HTTPS en entornos de producción)
├─ 5. Routing (Resolución de endpoints y rutas coincidentes)
├─ 6. NexusCors (Política de CORS adaptativa para clientes web autorizados)
├─ 7. Authentication (Extracción y verificación criptográfica de JWT / Google OAuth)
├─ 8. Authorization (Validación de políticas y permisos [Authorize])
├─ 9. Action Filters:
│   └─ ValidationFilter (Intercepta ModelState y ejecuta validadores FluentValidation)
│   └─ RequirePermissionAttribute (Verifica privilegios RBAC específicos de módulo y acción)
└─ 10. Controller Action / SignalR Hub (Ejecución de la lógica del caso de uso)
```

Manejo Global de Excepciones (GlobalExceptionHandler)

Toda excepción lanzada es interceptada por `GlobalExceptionHandler.cs`, transformándola en una respuesta estándar RFC 7807 (**ProblemDetails**) sin exponer volcados de pila sensibles:

Tipo de Excepción	Código HTTP	Semántica y Formato de Respuesta
ValidationException	400 BAD REQUEST	Fallo de validación en DTO. Devuelve un diccionario estructurado <code>errors: { "campo": ["mensaje"] }</code> .
UnauthorizedAccessException	401 UNAUTHORIZED	Credenciales incorrectas, usuario inactivo, cuenta bloqueada o token revocado/expirado.
ForbiddenException	403 FORBIDDEN	Usuario autenticado cuyo rol carece del permiso requerido para el módulo o acción solicitada.
NotFoundException	404 NOT FOUND	El recurso solicitado (paciente, cita, odontograma, usuario) no existe en la base de datos.
ConflictException	409 CONFLICT	Conflicto de unicidad (documento de identidad ya registrado, correo duplicado o Google ya vinculado).
BusinessRuleException	422 UNPROCESSABLE	Violación de regla clínica (ej. desvincular Google sin haber definido contraseña previa).
Exception (No controlada)	500 SERVER ERROR	Error interno del servidor. Se registra en Serilog con nivel Error y retorna un mensaje genérico.

Serialización Personalizada de Fechas y Horas

ASP.NET Core incluye serializadores propios `TimeOnlyJsonConverter` para formatear franjas horarias clínicas (`"HH:mm"`) sin desfases de zona horaria entre Oracle y React.

6. Modelo de Identidad y Autenticación

El sistema separa estrictamente cuatro conceptos de identidad médica y de acceso:

UserId (Acceso) Credenciales de login, hash BCrypt, estado activo y roles asignados.	PersonId (Legal) Identidad legal: tipo y número de documento, nombres, apellidos, teléfono.	PatientId (Clínica) Expediente odontológico, antecedentes, odontogramas y evolución médica.	ProfessionalId (Médica) Ficha profesional del odontólogo, número de licencia y especialidades.
--	---	---	--

7. Cuentas y Credenciales Oficiales de Prueba (Seeded Accounts)

Para la sustentación y evaluación en vivo por parte del jurado, se encuentran habilitadas las siguientes cuentas oficiales de prueba en la base de datos central. Se puede acceder indistintamente mediante el **Número de Documento** o el **Correo Electrónico**:

Rol del Sistema	Nombre del Usuario	Documento (Login)	Correo Electrónico	Contraseña Demo
Administrador (AD001)	Carlos Administrator	1234567890	admin@nexusodonto.com	Admin123!
Odontólogo (OD001)	Dra. Laura Gómez	1098765432	odontologo@nexusodonto.com	Dentist123!
Odontólogo (OD002)	Dr. Roberto Martínez	1098765433	roberto.martinez@nexusodonto.com	Dentist123!
Recepcionista (RC001)	María Rodríguez	1087654321	recepcion@nexusodonto.com	Reception123!
Paciente (PA001)	Andrés Pérez	1076543210	paciente@nexusodonto.com	Patient123!

Emisión y Rotación de Tokens JWT El token de acceso (<code>accessToken</code>) tiene una vigencia de 60 minutos con claims esenciales (<code>sub</code> , <code>roleId</code> , <code>personId</code>). El <code>refreshToken</code> viaja protegido y se rota criptográficamente en cada llamada a <code>POST /api/auth/refresh</code> .	Revocación Inmediata (POST /api/auth/logout) Al invocar el endpoint de cierre de sesión, la sesión se marca en la base de datos como <code>Revoked</code> con la marca de tiempo exacta (<code>RevokedAt</code>). Esto impide que el refresh token sea utilizado nuevamente, garantizando un cierre de sesión definitivo.
---	---

8. Control de Acceso Basado en Roles (RBAC) y Permisos

NexusOdonto implementa un esquema **RBAC** granular. Cada controlador decora sus métodos con el filtro `[RequirePermission(Module, Action)]`. La autorización verifica dinámicamente si el rol del usuario posee asignado el permiso correspondiente en la base de datos:

Módulo del Sistema	Administrador	Odontólogo	Recepcionista	Paciente	Acciones Soportadas
USERS / ROLES	Total (V, C, E, D)	Sin acceso	Sin acceso	Sin acceso	View, Create, Edit, Delete
PATIENTS	Total (V, C, E, D)	Lectura / Edición	Total (V, C, E)	Propio (V, E)	View, Create, Edit, Delete
PROFESSIONALS	Total (V, C, E, D)	Lectura (V)	Lectura (V)	Lectura (V)	View, Create, Edit, Delete
APPOINTMENTS	Total (V, C, E, D)	Agenda Propia (V, E)	Total (V, C, E, D)	Propias (V, C)	View, Create, Edit, Delete
AVAILABILITIES	Total (V, C, E, D)	Gestión Propia (V, E)	Lectura (V)	Lectura (V)	View, Create, Edit, Delete
ODONTOGRAMS	Total (V, C, E, D)	Clínico Total (V, C, E)	Sin acceso	Lectura Propia (V)	View, Create, Edit, Delete
CLINICAL_HISTORIES	Total (V, C, E, D)	Clínico Total (V, C, E)	Crear preliminar	Lectura Propia (V)	View, Create, Edit, Delete
SERVICES & CATALOGS	Total (V, C, E, D)	Lectura (V)	Lectura (V)	Lectura (V)	View, Create, Edit, Delete
AUDIT	Lectura Total (V)	Sin acceso	Sin acceso	Sin acceso	View

Mecanismo de Verificación en Tiempo de Ejecución

```
// Ejemplo real en controlador:
[HttpPost]
[RequirePermission("APPOINTMENTS", PermissionAction.Create)]
public async Task<ActionResult<AppointmentResponseDto>> Create([FromBody] CreateAppointmentDto request)
{
    // 1. El filtro extrae los roles y permisos del token ClaimsPrincipal
    // 2. Si el usuario carece del permiso, retorna 403 Forbidden antes de ejecutar el servicio
    // 3. Si está autorizado, ejecuta el caso de uso con garantía de integridad
}
```

9. Catálogo Completo de Endpoints REST

La API expone un catálogo exhaustivo de endpoints RESTful organizados bajo controladores fuertemente tipados:

Método	Ruta del Endpoint	Módulo / Permiso	Descripción de la Operación
POST	/api/auth/login	Público (Anónimo)	Inicio de sesión con documento/correo y contraseña. Retorna JWT y refresh token.
POST	/api/auth/register	Público (Anónimo)	Registro y onboarding integral de paciente nuevo con creación de credenciales.
POST	/api/auth/register-google	Público (Anónimo)	Registro acelerado de paciente mediante federación Google OAuth 2.0.
GET	/api/auth/google/login	Público (Anónimo)	Inicia el desafío OAuth contra Google y redirige a la página de consentimiento.
POST	/api/auth/google/link	Público (LinkToken)	Vincula una cuenta de Google a un usuario existente mediante verificación de credenciales.
DELETE	/api/auth/google/unlink	Autenticado	Desvincula la cuenta de Google (valida que el usuario tenga contraseña asignada).
POST	/api/auth/refresh	Público (Token)	Renovación de token de acceso mediante refresh token con rotación de clave.
POST	/api/auth/logout	Público (Token)	Cierre de sesión y revocación inmediata del refresh token en Oracle Database.
GET	/api/auth/me	Autenticado	Consulta el perfil completo, roles y permisos del usuario autenticado actual.
PUT	/api/auth/me	Autenticado	Actualización de datos filiatorios personales del usuario logueado.
POST	/api/auth/change-password	Autenticado	Cambio de contraseña con verificación de longitud mínima y hashing BCrypt.
GET	/api/auth/register-options	Público (Anónimo)	Lista tipos de documento y sexos biológicos activos para el formulario de registro.
GET	/api/v1/Appointments	APPOINTMENTS (View)	Listado de citas médicas con soporte de filtros por fecha, paciente y doctor.
POST	/api/v1/Appointments	APPOINTMENTS (Create)	Agendamiento de cita médica con validación de disponibilidad y conflicto horario.
PUT	/api/v1/Appointments/{id}	APPOINTMENTS (Edit)	Modificación de detalles, reprogramación de horario o cambio de odontólogo.
DELETE	/api/v1/Appointments/{id}	APPOINTMENTS (Delete)	Cancelación o eliminación asistida de una cita médica registrada.
GET	/api/v1/Appointments/requires-reschedule	APPOINTMENTS (View)	Obtiene citas futuras que entraron en conflicto por cambios de jornada laboral.
GET	/api/v1/Availabilities	AVAILABILITIES (View)	Consulta de turnos, jornadas y horarios semanales por consultorio y profesional.
POST	/api/v1/Availabilities/impact	AVAILABILITIES (View)	Simula y evalúa el impacto de un cambio de horario sobre citas ya programadas.
PUT	/api/v1/Availabilities/professional/{id}	AVAILABILITIES (Edit)	Actualiza la jornada semanal completa del odontólogo (aplica a partir de mañana).
GET	/api/v1/ClinicalHistories/patient/{id}	CLINICAL_HISTORIES	Expediente clínico integral del paciente con antecedentes, atenciones y diagnósticos.
POST	/api/v1/ClinicalAttentions	CLINICAL_HISTORIES	Registro de evolución médica de consulta vinculada a una cita, diagnósticos y receta.
POST	/api/v1/Diagnoses	CLINICAL_HISTORIES	Registro de diagnóstico odontológico codificado bajo estándar internacional CIE-10.
POST	/api/v1/ProceduresPerformed	CLINICAL_HISTORIES	Registro de procedimientos clínicos ejecutados en el paciente durante la atención.
GET	/api/v1/PatientAntecedents/patient/{id}	CLINICAL_HISTORIES	Consulta de antecedentes médicos, alergias, patologías de base y medicación.
GET	/api/v1/Odontograms/patient/{id}/latest	ODONTOGRAMS (View)	Obtiene el estado más reciente del odontograma del paciente con todas sus piezas.

Método	Ruta del Endpoint	Módulo / Permiso	Descripción de la Operación
POST	/api/v1/0dontograms/full	ODONTOGRAMS (Create)	Guardado atómico del odontograma completo (piezas dentales, superficies y hallazgos).
GET	/api/v1/Teeth	ODONTOGRAMS (View)	Catálogo maestro anatómico de las 52 piezas dentales en estándar internacional FDI.
GET	/api/v1/DentalFindings	ODONTOGRAMS (View)	Listado de hallazgos clínicos dentales (caries, obturaciones, coronas, implantes).
GET	/api/v1/Patients	PATIENTS (View)	Directorio de pacientes con datos de contacto, documento y estado de expediente.
GET	/api/v1/Professionals	PROFESSIONALS (View)	Directorio de odontólogos con licencias, cargos, especialidades y jornadas.
GET	/api/v1/ClinicSettings	SETTINGS (View)	Consulta de configuración global de la clínica (horarios, receso de almuerzo, sillones).
PUT	/api/v1/ClinicSettings	SETTINGS (Edit)	Actualización de parámetros operativos de la clínica con persistencia central.
GET	/api/v1/HubNotifications	NOTIFICATIONS (View)	Bandeja de notificaciones internas de usuario con filtrado de leídas y no leídas.
GET	/api/v1/SupportTickets	SUPPORT (View)	Bandeja de tickets de soporte asistencial clasificados por motivo y nivel de prioridad.
GET	/api/v1/AuditEvents	AUDIT (View)	Trazabilidad inmutable de eventos de seguridad y modificaciones críticas en Oracle.
GET	/api/v1/Services	CATALOGS (View)	Catálogo oficial de servicios odontológicos, tarifas en COP y duración en sillón.
GET	/api/v1/[26_Catalogos_Base]	CATALOGS (View)	Controladores CRUD base para los 26 catálogos normalizados del sistema.

10. Reglas de Negocio Asistenciales y Lógica de Agenda

1. Política de Integridad: "Los Cambios de Horario Rigen a Partir de Mañana"

Fundamento Ético y Clínico de Integridad Asistencial

Si un odontólogo modifica su jornada laboral durante el día presente, el sistema **nunca cancela ni altera las citas de los pacientes que ya se encuentran citados o en sala de espera ese mismo día**. Los cambios de disponibilidad aplican estrictamente a partir de las 00:00 horas del día siguiente.

Para las citas futuras que entren en conflicto, el backend ejecuta una **evaluación de impacto** (`POST /api/v1/Availabilities/impact`) y las etiqueta con `requiresReschedule: true` , permitiendo a recepción coordinar la reasignación de manera asistida.

2. Asignación Automática de Odontólogo Alternativo

Cuando ocurre un conflicto de horario o una cita de urgencia, `AppointmentService.cs` ejecuta un algoritmo de búsqueda:

- Identifica odontólogos con la misma especialidad requerida que tengan disponibilidad libre en esa franja.
- Verifica que el consultorio o sillón cuente con capacidad física disponible.
- Asigna automáticamente al odontólogo libre más idóneo y notifica al personal vía SignalR.

3. Zona Horaria Oficial de la Clínica (ClinicTimeZone)

Para evitar discrepancias entre la nube y la sede física, el backend estandariza la hora oficial mediante `AppointmentService.GetClinicNow()` , resolviendo la zona horaria **America/Bogota (UTC-5 / SA Pacific Standard Time)**. Todas las validaciones de fechas pasadas y cálculos de turnos se realizan contra la hora clínica real.

4. Protección de Franjas de Almuerzo y Duración de Sillón

Bloqueo de Almuerzo Clínico

El sistema bloquea automáticamente la franja de descanso (12:00 a 14:00), impidiendo que se agenden consultas sobre las horas de almuerzo.

Duración Mínima de Procedimientos

Cada servicio del catálogo especifica su duración en sillón (15, 30, 60 o 90 minutos). El agendamiento calcula dinámicamente `EndsAt = StartsAt + Duration` .

11. Odontograma Anatómico FDI y Expediente Clínico

NexusOdonto implementa el estándar internacional de la **Federación Dental Internacional (FDI)** con soporte completo para dentición adulta (32 piezas) y dentición temporal/infantil (20 piezas):

Dentición Permanente (32 Piezas Adultas)

Cuadrante 1 (18 a 11), Cuadrante 2 (21 a 28), Cuadrante 3 (38 a 31), Cuadrante 4 (41 a 48).

Dentición Decidua (20 Piezas Temporales)

Cuadrante 5 (55 a 51), Cuadrante 6 (61 a 65), Cuadrante 7 (75 a 71), Cuadrante 8 (81 a 85).

Modelo Vectorial de 5 Superficies Anatómicas

Cada diente se subdivide en 5 superficies anatómicas independientes:

- **Vestibular (V):** Cara externa orientada hacia los labios o mejillas.
- **Lingual / Palatino (L/P):** Cara interna orientada hacia la lengua o paladar.
- **Mesial (M):** Cara proximal orientada hacia la línea media de la arcada dental.
- **Distal (D):** Cara proximal orientada hacia la parte posterior de la arcada.
- **Oclusal / Incisal (O/I):** Superficie masticatoria o borde cortante.

Persistencia Atómica del Odontograma (POST /api/v1/Odontograms/full)

NexusOdonto realiza la persistencia en una **única transacción atómica**. Si el paciente cuenta con un odontograma previo, el backend actualiza los estados de los dientes existentes e inserta los nuevos hallazgos en un solo bloque confirmado por `SaveChangesAsync()`, garantizando la consistencia absoluta de la historia clínica.

Diagnósticos Codificados CIE-10 y Prescripciones Médicas

Cada atención clínica registrada en `POST /api/v1/ClinicalAttentions` se vincula a una cita y almacena:

- **Motivo de consulta y evolución médica:** Anamnesis detallada y observaciones del profesional.
- **Diagnósticos CIE-10:** Código internacional estandarizado (ej. *K02.1 Caries de la dentina*, *K05.0 Gingivitis aguda*).
- **Procedimientos Realizados:** Procedimientos clínicos ejecutados vinculados al tarifario oficial.
- **Prescripción Farmacológica:** Medicamentos recetados con dosis, frecuencia, vía de administración y duración.

12. Comunicación en Tiempo Real (SignalR) y Presencia

El backend integra **Microsoft SignalR Core** a través del Hub central `/hubs/notifications` (`NexusNotificationHub.cs`), proporcionando comunicación bidireccional continua con baja latencia mediante WebSockets:

Autenticación en WebSocket

La conexión WebSocket se autentica mediante token JWT enviado en el parámetro `access_token` del query string o mediante la cabecera `Authorization: Bearer`.

Suscripción por Grupos de Rol

Al conectarse, `OnConnectedAsync` suscribe automáticamente la conexión a los grupos de sus roles (`Groups.AddToGroupAsync(connectionId, roleId)`), permitiendo emitir alertas dirigidas.

Rastreador de Presencia Concurrente en Memoria (PresenceTracker)

El servicio singleton `IPresenceTracker` mantiene el estado en línea de los odontólogos y personal mediante estructuras concurrentes seguras:

- Soporte Multi-Pestaña:** Gestiona múltiples `ConnectionIds` por usuario. El evento `UserOnline` se emite al conectar la primera pestaña, y `UserOffline` solo cuando se cierra la última conexión.
- Snapshot Inicial de Presencia:** Al conectar, el cliente recibe la lista completa de usuarios en línea mediante `Clients.Caller.PresenceSnapshot(onlineUsers)`.

Catálogo de Eventos SignalR Transmitidos

Evento SignalR	Destinatarios	Propósito y Carga Útil (Payload)
UserOnline	Clients.Others	Notifica que un odontólogo o personal se conectó al sistema. Incluye datos de perfil y rol.
UserOffline	Clients.Others	Notifica la desconexión del usuario para actualizar el halo de presencia en el panel clínico.
PresenceSnapshot	Clients.Caller	Envía la lista consolidada de todo el personal conectado al momento del ingreso.
ReceiveNotification	Usuario o Grupo	Envía una notificación institucional o alerta clínica directa al destinatario.
AppointmentCreated	Odontólogo / Recepción	Difunde la creación de una nueva cita médica en tiempo real para refrescar agendas activas.
AppointmentUpdated	Odontólogo / Recepción	Difunde cambios de estado, reprogramaciones o cancelaciones de citas médicas.

13. Persistencia Oracle Database, Transacciones y Seeders

El acceso a datos se gestiona con **EF Core 10** y el proveedor **Oracle.EntityFrameworkCore**:

Convención snake_case y Minúsculas Citadas

Oracle trata los identificadores entrecomillados como sensibles a mayúsculas. Todas las tablas y columnas se mapean en `snake_case` y minúsculas citadas (`"patients"` , `"appointments"`).

Claves Primarias Determinísticas (Guid)

Los registros maestros utilizan identificadores `Guid` fijos y canónicos, garantizando que los seeders sean inmutables y no produzcan duplicaciones.

El patrón `IUnitOfWork` centraliza el método `SaveChangesAsync()` , asegurando que múltiples modificaciones en diferentes agregados se confirmen bajo una **única transacción ACID** en Oracle.

14. Value Objects y Diseño Guiado por el Dominio (DDD)

Para evitar la obsesión por tipos primitivos, la capa de dominio encapsula reglas de validación en **Value Objects** inmutables:

Value Object	Tipo Base	Reglas de Validación y Semántica de Negocio
CatalogCode	string	Código alfanumérico en mayúsculas (2 a 20 caracteres) sin espacios ni caracteres especiales.
Price	decimal	Valor monetario no negativo en pesos colombianos (COP). Validación de límite y 2 decimales.
HireDate	DateOnly	Fecha de contratación laboral válida. No permite fechas futuras ni anteriores a la clínica.
ProfessionalLicense	string	Número de registro médico y tarjeta profesional odontológica (máximo 50 caracteres).
PhoneNumber	string	Número telefónico normalizado en formato internacional (+57 o estándar E.164).
TimeOfDay	TimeOnly	Hora del día en intervalos estándar de consulta médica (minutos en múltiplos de 5).
RagConfidence	decimal	Nivel de confianza probabilística normalizado entre 0.0 y 1.0 para respuestas automáticas.

15. Validaciones Declarativas y Resiliencia HTTP con Polly

La capa de aplicación utiliza **FluentValidation** para la validación declarativa de contratos antes de invocar casos de uso. La comunicación con servicios externos (WhatsApp / Evolution API) utiliza **Microsoft.Extensions.Http.Resilience**:

Reintentos Exponenciales (Backoff)

Ante un fallo de red transitorio (códigos 5xx o timeout), el cliente reintenta automáticamente la petición 3 veces con demoras crecientes (2s, 4s, 8s).

Disyuntor (Circuit Breaker)

Si el servicio externo falla de manera sostenida durante más de 30 segundos, el disyuntor abre el circuito para evitar saturar el servidor.

16. Seguridad, Confidencialidad y Enmascaramiento de Datos

En cumplimiento de los estándares de seguridad de software y la legislación de protección de datos médicos, el backend aplica un protocolo estricto de protección de secretos:

Protocolo de Confidencialidad de Infraestructura

Las credenciales oficiales de prueba para evaluación académica se encuentran disponibles en la **Sección 7** (`Admin123!` , `Dentist123!` , etc.). Todas las llaves criptográficas de producción, secretos de proveedores OAuth y cadenas de conexión a base de datos están estrictamente enmascaradas en la documentación:

Variable / Secreto	Función en el Sistema	Formato de Enmascaramiento Documental
ORACLE_CONNECTION_STRING	Cadena de conexión al servidor de base de datos Oracle 21c/23ai.	(aquí va la cadena de conexión / Data Source=localhost:1521/XEPDB1;UserId=...;Password=...)
Jwt__SigningKey	Clave secreta simétrica de al menos 32 bytes para firma HMAC-SHA256 de tokens JWT.	(aquí va la clave secreta JWT / SigningKey de al menos 32 caracteres para firma segura)
Google__ClientSecret	Secreto de cliente OAuth 2.0 emitido por Google Cloud Console.	(aquí va el secreto del cliente Google OAuth / ...)
EvolutionApi__ApiKey	Token de autenticación para la pasarela de mensajería WhatsApp.	(aquí va la API key del servidor de mensajería / ...)
X-Internal-Secret	Cabecera secreta para comunicación entre microservicios autorizados.	(aquí va la clave secreta interna de comunicación / ...)

17. Guía Técnica para Sustentación ante Jurados

Preguntas técnicas de alta complejidad preparadas para responder con total solvencia durante la evaluación y sustentación del backend:

1. ¿Por qué se utilizó Clean Architecture en lugar de una arquitectura clásica de 3 capas?

Respuesta para el jurado: Porque Clean Architecture implementa el *Principio de Inversión de Dependencias (DIP)*. En una arquitectura tradicional, la lógica de negocio depende directamente de la base de datos (EF Core). En Clean Architecture, la capa de **Domain** es 100% pura y agnóstica; la capa de **Application** define interfaces de repositorio (`IAppointmentRepository`), y es **Infrastructure** quien implementa los adaptadores Oracle. Esto permite cambiar de motor de base de datos, simular pruebas unitarias o integrar servicios externos sin alterar una sola línea de las reglas de negocio odontológicas.

2. ¿Cómo maneja el backend la concurrencia cuando dos pacientes intentan agendar la misma cita simultáneamente?

Respuesta para el jurado: Se maneja en dos niveles concéntricos: primero, `AppointmentService.cs` ejecuta una consulta de validación cruzada con bloqueo y verificación de solapamiento de intervalos (`StartsAt < endsAt && EndsAt > startsAt`) dentro de la misma transacción de Unit of Work. Segundo, a nivel de Oracle, la tabla `appointments` cuenta con restricciones de integridad relacional. Si ocurre un intento concurrente, la transacción conflictiva falla de forma controlada y retorna un error 409 (Conflict) sin corromper la agenda.

3. ¿Cómo garantiza el sistema que las notificaciones de SignalR lleguen únicamente al usuario correcto y no se filtren a otros?

Respuesta para el jurado: El Hub `NexusNotificationHub` implementa `CustomUserIdProvider` , que asocia cada conexión WebSocket con el `ClaimTypes.NameIdentifier` (UserId) del token JWT verificado. Al emitir una notificación, el servicio `NotificationPublisherService` utiliza `Clients.User(targetUserId)` o `Clients.Group(roleId)` . El protocolo WebSocket de SignalR aísla los canales a nivel de transporte, garantizando privacidad absoluta entre pacientes y profesionales.

4. ¿Por qué se optó por Oracle Database y qué desafíos implicó su integración con EF Core?

Respuesta para el jurado: Oracle Database 21c/23ai ofrece robustez empresarial, transaccionalidad ACID estricta y soporte para altos volúmenes clínicos. El principal desafío técnico radicó en la sensibilidad de mayúsculas/minúsculas en identificadores citados de Oracle. Se resolvió configurando todas las entidades y columnas en `snake_case` minúsculas con identificadores canónicos citados (`"patients"` , `"first_name"`) y utilizando claves primarias `Guid` determinísticas para garantizar la inmutabilidad de los seeders y migraciones.

18. Referencias Cruzadas y Mapeo de Código Fuente

Estructura de archivos y directorios en el repositorio `NexusOdontoBackend_Api` para verificación directa de los jurados:

Componente / Capa	Archivos Clave en el Repositorio	Responsabilidad
Controladores REST	<code>Api/Controllers/Auth/AuthController.cs</code> <code>Api/Controllers/Schedule/AppointmentsController.cs</code> <code>Api/Controllers/Clinical/ClinicalHistoriesController.cs</code> <code>Api/Controllers/Odontogram/OdontogramsController.cs</code>	Puntos de entrada HTTP y enrutamiento RESTful.
Hub SignalR & Presencia	<code>Api/Hubs/NexusNotificationHub.cs</code> <code>Api/Providers/CustomUserIdProvider.cs</code> <code>Application/Services/Presence/PresenceTracker.cs</code>	Gestión de conexiones WebSocket y presencia en vivo.
Middlewares & Filtros	<code>Api/Middleware/GlobalExceptionMiddleware.cs</code> <code>Api/Filters/RequirePermissionAttribute.cs</code> <code>Api/Filters/ValidationFilter.cs</code>	Manejo de excepciones RFC 7807 y seguridad RBAC.
Servicios de Aplicación	<code>Application/Services/AuthService.cs</code> <code>Application/Services/Schedule/AppointmentService.cs</code> <code>Application/Services/Clinical/ClinicalAttentionService.cs</code> <code>Application/Services/Odontogram/OdontogramService.cs</code>	Orquestación de casos de uso y lógica de negocio.
Entidades & Value Objects	<code>Domain/Entities/People/UserEntity.cs</code> <code>Domain/Entities/Schedule/AppointmentEntity.cs</code> <code>Domain/Entities/Odontogram/OdontogramEntity.cs</code> <code>Domain/ValueObjects/*.cs</code>	Modelos de datos puros y tipos de valor de dominio.
Persistencia & Contexto	<code>Infrastructure/Persistence/Context/AppDbContext.cs</code> <code>Infrastructure/Persistence/Context/DatabaseInitializer.cs</code> <code>Infrastructure/UnitOfWorks/UnitOfWork.cs</code> <code>Infrastructure/Repositories/**/*cs</code>	Acceso a Oracle EF Core y transacciones ACID.

Declaración de Calidad y Finalización Técnica
El backend de NexusOdonto cumple con el 100% de los requerimientos de arquitectura limpia, seguridad, rendimiento y buenas prácticas de ingeniería de software. Compilado con 0 errores y 0 advertencias bajo .NET 10.